

Reviewer Pack — Minimal Order of Local Systems in Algebraic Geometry and Res...

Entry 1fe18d27f2fc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 18:57:21 UTC

Minimal Order of Local Systems in Algebraic Geometry and Resolution Proof Length

Entry ID: 1fe18d27f2fc **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 18:57:21 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Local Systems) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

The length of the shortest resolution proof for a SAT instance is lower-bounded by the minimal order of a local system associated with the instance's incidence algebra, i.e., $L(\varphi) = \Theta(\text{OInc}(\varphi))$, where $L(\varphi)$ is the resolution proof length and $\text{OInc}(\varphi)$ is the order of the incidence algebra's local system.

Rationale (proposer's reasoning):

Local systems in algebraic geometry provide a generalization of the concept of dimension, capturing the complexity of geometric structures. By relating this to resolution proof length, we may uncover new connections between geometric properties and computational complexity, potentially leading to improved lower bounds for resolution proofs.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 81b6de060078db44

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the instances have a Pearson's correlation coefficient between $\text{OInc}(\varphi)$ and $L(\varphi)$ greater than or equal to 0.5, and no instance has a resolution proof length less than half its associated incidence algebra's local system order.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebraic geometry AND local systems AND resolution proof complexity
- resolution proof length AND incidence algebra AND minimal order - Boolean satisfiability
AND SAT instance AND minimal order of local systems

Top relevant hits considered: - [http://arxiv.org/abs/1206.1925v1] Counting Algebraic Curves with Tropical Geometry - [http://arxiv.org/abs/2111.12700v2] Universality in long-distance geometry and quantum complexity - [http://arxiv.org/abs/math/0106084v1] How to show a set is not algebraic - [http://arxiv.org/abs/astro-ph/0208243v7] Measurement of the Flux of Ultrahigh Energy Cosmic Rays from Monocular Observations by the High Resolution Fly's Eye Exp - [http://arxiv.org/abs/astro-ph/0407622v3] A Study of the Composition of Ultra High Energy Cosmic Rays Using the High Resolution Fly's Eye - [http://arxiv.org/abs/1904.12405v1] Resolutions of length four which are Differential Graded Algebras - [http://arxiv.org/abs/1004.0653v2] Exact Ramsey Theory: Green-Tao numbers and SAT - [http://arxiv.org/abs/2605.09347v1] Dsat: A Native SAT Solver for Discrete Logic

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.3s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_sat_instance(n):
        variables = set()
        clauses = []
        for _ in range(2 * n):
            clause = [random.choice([f'x{i}', f'-x{i}']) for i in range(1, n + 1)]
            clauses.append(clause)
            variables.update(clause)
        return list(variables), clauses

    def incidence_algebra(clauses):
        n = len(set(var[2:] for var in sum(clauses, [])))
        A = [[0] * (n + 1) for _ in range(n + 1)]
        for clause in clauses:
            for literal in clause:
                if literal.startswith('x'):
                    i = int(literal[1:]) - 1
```

```

        else:
            i = int(literal[2:]) - 1
            A[i][i] += 1
    return A

def matrix_rank(A):
    m, n = len(A), len(A[0])
    rank = 0
    for col in range(n):
        pivot_row = None
        for row in range(rank, m):
            if A[row][col] != 0:
                pivot_row = row
                break
        if pivot_row is not None:
            A[pivot_row], A[rank] = A[rank], A[pivot_row]
            rank += 1
            for row in range(m):
                if row != rank - 1:
                    factor = A[row][col] / A[rank - 1][col]
                    for j in range(n):
                        A[row][j] -= factor * A[rank - 1][j]
    return rank

def local_system_order(A):
    rank = matrix_rank(A)
    n = len(A) - 1
    if rank != n:
        raise ValueError("Matrix is singular")
    order = Fraction(1, math.factorial(n))
    for i in range(n + 1):
        for j in range(i + 1, n + 1):
            order *= Fraction(A[i][j], A[j][i])
    return order

def resolution_length(clauses):
    stack = []
    for clause in clauses:
        if not any(literal in stack for literal in clause):
            stack.append(random.choice(clause))
    length = len(stack)
    while stack:
        literal = stack.pop()
        new_clauses = [c for c in clauses if literal not in c and '-x' + literal not in c]
        if not new_clauses:
            break
        stack.extend(new_clause for clause in new_clauses if literal not in clause and '-x' + literal not in clause)
        length += len(stack)
    return length

variables, clauses = generate_sat_instance(40)
A = incidence_algebra(clauses)
try:
    OInc_phi = local_system_order(A)
except ValueError as e:
    return {

```

```

        "metric_name": "Local System Order",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": 40,
        "conjecture_holds": False,
        "counterexample": str(e)
    }

L_phi = resolution_length(clauses)

return {
    "metric_name": "Resolution Proof Length",
    "metric_value": L_phi,
    "instances_tested": 1,
    "n_max": 40,
    "conjecture_holds": L_phi >= 0Inc_phi / 2,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 97) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results if result["metric_value"] is not None)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results if result["metric_value"] is not None) / len(results))
    support_fraction = sum(result["conjecture_holds"] for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Resolution proof length less than half local system order\")
    else:
        print("RESULT: INCONCLUSIVE insufficient support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

is singular'}
TRIAL: {'metric_name': 'Local System Order', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
TRIAL: {'metric_name': 'Local System Order', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
TRIAL: {'metric_name': 'Local System Order', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
TRIAL: {'metric_name': 'Local System Order', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
TRIAL: {'metric_name': 'Local System Order', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
TRIAL: {'metric_name': 'Local System Order', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
TRIAL: {'metric_name': 'Local System Order', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
TRIAL: {'metric_name': 'Local System Order', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}

```

TRIAL: {'metric_name': 'Local System Order', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
 TRIAL: {'metric_name': 'Local System Order', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
 TRIAL: {'metric_name': 'Local System Order', 'metric_value': None, 'instances_tested': 1, 'n_max': 40}
 RESULT: FALSIFIED counterexample="Resolution proof length less than half local system order" first_fa

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code only tests instances with $n = 40$, which is too small to be conclusive. The metric 'Local System Order' may not scale trivially with n , and the failure of the conjecture at this size does not necessarily imply it will fail for larger values of n .

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that the conjecture does not hold for at least one instance, as shown by a counterexample where the resolution proof length | next: Further investigation is needed to determine if this counterexample holds for larger values of n or if there are other factors affecting the validity of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14665
2	preregistration	ollama_remote	glm4:latest	0	0	14243
3	novelty	ollama_remote	glm4:latest	0	0	8298
4	novelty	ollama_remote	glm4:latest	0	0	11450
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17570
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15269
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40307
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15718
9	critic	ollama_remote	glm4:latest	0	0	66789
10	judge	ollama_remote	glm4:latest	0	0	10070

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 214378 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/1fe18d27f2fc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1fe18d27f2fc.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/1fe18d27f2fc.tar.gz (if generated)